

ToCo-Split: Topology-Constrained Communication for Efficient Split Learning in Edge Intelligence

Nasif Fahmid Prangon and Jie Wu

Department of Computer and Information Sciences, Temple University, USA

Abstract—Distributed learning over edge networks requires balancing computation, communication, and statistical efficiency under heterogeneous system conditions. Federated learning (FL), split learning (SL), and SplitFed learning (SFL) address this problem through different computation and synchronization mechanisms, but they often depend on centralized coordination that limits scalability and introduces synchronization overhead. In this paper, we present an SL-based system considering three key elements: cut-layer identification, client clustering, and communication topology design. Cut layers determine where computation should be partitioned to balance computation and communication; clients are then grouped according to similar statistics of their local datasets without sharing the datasets themselves; finally, topology structures peer-to-peer communication in a manner that differs from SFL, where full client-side aggregation is typically required. This design extends SL with a topology-dependent gradient mixing scheme, in which clients perform forward propagation locally, send their computed activations to the central server, receive gradients from the server, and mix those gradients through the topology structure before updating the local model. From a systems perspective, this paper explores how structured topology can approximate full client-side coordination through decentralized communication among neighboring nodes. We further examine how topology, workload, and non-IID data affect communication and learning efficiency in edge environments.

Index Terms—Communication efficiency, distributed learning, edge intelligence, heterogeneous networks, split learning, topology-aware learning.

I. INTRODUCTION

The fast development of edge intelligence makes distributed learning among networked nodes an essential systems problem. In some scenarios, edge nodes collaboratively learn a common model without transferring data to other machines. Federated learning (FL) can help with privacy-preserving learning by supporting clients' locally learning complete models and synchronously transferring the model update parameters to the central server [1]. But FL may suffer from problems such as system heterogeneity, non-IID dataset, and additional cost of communication [2]. Specifically, clients have different computational capabilities, network conditions, and availability, which leads to stragglers and synchronization costs [2]. Such issues become more prominent when edge learning involves many clients, in which case, communicating repeatedly with the server brings high coordination costs.

In order to further reduce computation on clients, split learning (SL) splits the model into two parts on both the server and clients, so that each client can perform forward computation on the front part of the model while offloading

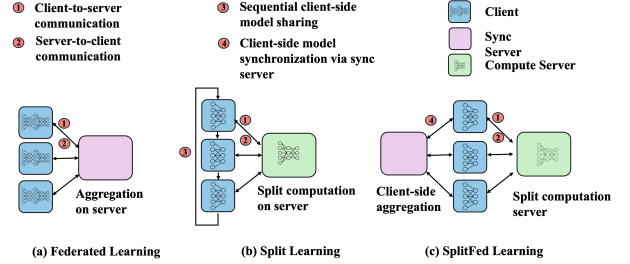


Fig. 1. Overview of federated learning (FL), split learning (SL), and SplitFed learning (SFL), highlighting their differences in model partitioning, communication flow, and client synchronization.

the rest of the computation to the server [3]. Traditional SL works in a sequential manner, where clients communicate with the server in a sequential way by forwarding activations from clients to the server via forward computation and sending cut-layer gradients to the clients through backward propagation. As illustrated in Fig. 1, FL, SL, and SplitFed Learning (SFL) all involve computation and synchronization in different ways. Specifically, FL uses complete local training and server aggregation, SL communicates activations and gradients of the cut layer, and SFL uses split computation and federated learning in client's model portion [4].

In this paper, we argue that efficient distributed learning requires joint design across three dimensions: *cut layer selection*, *client clustering*, and *communication topology*. Cut layer determines the balance between computation and communication resources, client clustering ensures efficient processing by grouping clients with homogeneous data distribution and mitigates negative impacts from non-IID distribution, and communication topology determines how information flows among clients. Although partitioning and clustering have been investigated, topology is commonly viewed as a constraint imposed by the system rather than a design factor.

Our key insight is that topology allows approximation of full client side coordination. Rather than performing all computations using global aggregation, it is feasible to propagate information over a certain topology including line, ring, and torus. These structures reduce coordination overhead while enabling controlled information mixing among neighboring clients, offering a flexible trade-off between communication cost and learning efficiency.

In light of this insight, we introduce a topology-aware distributed SL framework. Our proposal combines the standard server-side computation process of SL with commu-

nication processes constrained by communication topology on the client side. More specifically, our approach involves performing forward propagation locally, sending the activation data to the server, and receiving the computed gradients from the server. After obtaining the updated weights, clients share the gradients with neighbors to further improve their learning performance. By extending SL with topology-constrained gradient mixing, the proposed framework enables limited client-side coordination without full client-side aggregation.

Furthermore, we study how topology, workload, and non-IID data jointly affect communication and learning in edge environments.

The main contributions of this paper are summarized as follows:

- We propose a topology-aware SL framework that jointly considers cut layer selection, client clustering, and structured client-side communication for efficient distributed learning in heterogeneous edge networks.
- We introduce topology-based gradient mixing to approximate full client-side coordination using only local communication among neighboring clients.
- We analyze how topology, workload distribution, and data heterogeneity affect communication efficiency and learning behavior.

II. RELATED WORK

Distributed learning has become a cornerstone for large-scale collaborative model training across networked systems, enabling multiple agents to jointly optimize a global objective without centralizing raw data. Classical frameworks include both parameter-server-based architectures and fully decentralized optimization methods, with extensive studies on convergence guarantees under various system and data assumptions [5]–[7]. Despite these advances, such approaches incur substantial communication overhead due to iterative exchange of high-dimensional model updates, which becomes a critical bottleneck in wireless and resource-constrained environments [5].

To address this limitation, a significant body of work has focused on communication-efficient distributed learning. Existing techniques broadly fall into three categories: reducing communication rounds via local updates or event-triggered schemes [1], [8], compressing transmitted information through quantization and sparsification [9], [10], and optimizing wireless resource allocation and scheduling strategies [11]. These approaches have been extensively explored in federated learning (FL), where clients collaboratively train a global model while keeping data localized [1]. Although FL mitigates privacy concerns and leverages distributed computation, it continues to suffer from significant communication overhead due to repeated model aggregation, particularly as model sizes and system scales increase [12]. Furthermore, system heterogeneity, non-IID data distributions, and dynamic wireless conditions introduce additional challenges in achieving efficient and scalable training [12], [13].

To alleviate the computational burden on resource-constrained devices, split learning (SL) has emerged as an alternative paradigm that partitions deep models between clients and edge servers, enabling workload offloading while preserving data privacy [14], [15]. Extensions such as SplitFed Learning (SFL) and Parallel Split Learning (PSL) further improve scalability by enabling parallel training and hybrid aggregation mechanisms [14]. More advanced frameworks, such as EPSL, reduce server-side overhead by aggregating last-layer activation gradients prior to backpropagation while jointly optimizing wireless resource allocation and cut layer selection [16]. However, despite these improvements, existing approaches largely rely on fixed model partitioning and centralized coordination, overlooking the impact of communication structure and topology on system performance.

To overcome these limitations, recent works have explored fine-grained communication strategies. Layer-wise methods, such as FedLAMA and FedLUAR, adapt communication frequency based on layer importance, enabling efficient synchronization by prioritizing critical layers [17], [18]. In parallel, cut layer selection has been identified as a key design parameter in split learning, introducing a trade-off between communication cost and computational load [19]. Beyond system efficiency, cut placement also influences convergence behavior and model performance, particularly in SFL settings where shared server-side models affect global knowledge integration [20].

Complementing these directions, topology-aware decentralized learning has demonstrated that structured communication can significantly influence convergence and scalability. In such settings, properties such as spectral gap and graph connectivity determine how efficiently information propagates across clients, with structured topologies like hypercube and torus outperforming simple ring-based communication [21]. Moreover, topology design can be optimized to account for data heterogeneity, improving training stability and convergence speed [21].

Recent advances in SFL further highlight the importance of both learning dynamics and communication structure. Han *et al.* provide convergence guarantees for SFL under heterogeneous data, demonstrating its robustness compared to traditional FL approaches [22]. Meanwhile, Shen *et al.* propose RingSFL, which integrates split learning with ring-based communication to mitigate straggler effects and improve training efficiency [23].

Overall, existing works have made progress in optimizing communication efficiency, layer-wise adaptation, cut layer selection, and topology design. However, these directions are largely explored in isolation. In particular, prior work does not jointly consider how model partitioning (*where* computation occurs), layer-wise dynamics (*what* is communicated), and communication topology (*how* information flows) interact in split learning systems. In contrast, our proposed approach treats split learning as a joint design problem by combining cut layer selection, layer-aware communication behavior, and topology-guided client interaction within a single framework.

TABLE I
SYMBOLS AND NOTATIONS

Symbol	Description
$N, \mathcal{U}$	Total number of clients and set of all clients
$D_i, D_i $	Local dataset of client i and its size
C_i	Computational capability of client i
L, k	Total number of model layers and cut layer index
$S_k, S_k^{(g)}$	Activation and gradient sizes at layer k
$\text{Comp}_i(k)$	Computation cost of client i up to layer k
$\mathbf{p}_i, C$	Label distribution vector of client i and number of classes
$\text{Sim}(i, j)$	Similarity between clients i and j
$K, \mathcal{C}_m$	Number of clusters and the m -th client cluster
G_m, E_m	Topology graph of cluster m and its edge set
$\mathcal{N}(i)$	Neighbor set of client i in G_m
$g_i, \tilde{g}_i$	Server and refined gradient after intra-cluster mixing
w_{ij}	Gradient mixing weight between clients i and j
S_g	Size of exchanged gradients

By using structured topologies for client-side mixing, the framework approximates full coordination with lower overhead while preserving useful knowledge propagation across heterogeneous clients. This gap motivates the need for a unified framework that integrates these dimensions to achieve scalable and efficient distributed learning.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Model

We consider a distributed split learning system consisting of N clients, denoted by $\mathcal{U} = \{1, 2, \dots, N\}$, and a central server. Each client $i \in \mathcal{U}$ holds a local dataset D_i of size $|D_i|$ and has computational capability C_i . The global model consists of L layers.

Under the split learning framework, the model is partitioned at a cut layer $k \in \{1, 2, \dots, L\}$. The first k layers are executed locally at each client, while the remaining layers are processed at the server. During training, each client performs forward propagation up to the cut layer and transmits the resulting intermediate activations to the server. The server completes the forward pass, computes the loss, and performs backpropagation to obtain gradients with respect to the cut-layer activations. These gradients are then returned to the corresponding clients for local backpropagation.

To improve learning efficiency under heterogeneous and non-IID settings, clients are further organized into clusters, within which they can communicate over a predefined topology. Specifically, after receiving gradients from the server, clients perform intra-cluster communication by exchanging gradient information with neighboring clients and refining their updates through local aggregation. This enables topology-aware collaboration at the client side, reducing gradient variance and improving convergence without requiring full global coordination.

The system operates under the following practical constraints:

- *Heterogeneous resources*: Clients have varying computational capabilities and communication bandwidth.
- *Non-IID data*: Local datasets across clients follow different label distributions.

- *Communication constraints*: Frequent exchange of activations, gradients, and intra-cluster messages incurs significant communication cost.

B. Cut Layer Selection

The selection of the cut layer k determines how computation and communication are distributed between the client and the server during split learning. A shallow cut reduces client-side computation but increases activation transmission overhead, whereas a deeper cut reduces communication at the expense of higher client-side computation. Therefore, instead of selecting the cut layer heuristically, we evaluate candidate split points using a cost-aware formulation.

For a model with L layers, the cumulative client-side and server-side computation associated with cut layer k are defined as

$$\text{Comp}^{\text{client}}(k) = \sum_{\ell=1}^k \text{FLOPs}_{\ell}, \quad (1)$$

and

$$\text{Comp}^{\text{server}}(k) = \sum_{\ell=k+1}^L \text{FLOPs}_{\ell}, \quad (2)$$

respectively, where FLOPs_{ℓ} denotes the computation required at layer ℓ .

Let S_k denote the size of the intermediate activations transmitted from the client to the server at the cut layer, and let $S_k^{(g)}$ denote the size of the corresponding gradients transmitted back to the client during backpropagation. Following standard split learning implementations, the gradient tensor at the cut layer has the same dimensionality as the corresponding activation tensor.

To account for heterogeneous client resources, computation and communication costs are expressed in time units. For client i , the client-side computation time is

$$T_i^{\text{client}}(k) = \text{Comp}^{\text{client}}(k) / R_i^{\text{client}}, \quad (3)$$

where R_i^{client} denotes the processing capability of client i . Similarly, the server-side computation time is

$$T^{\text{server}}(k) = \text{Comp}^{\text{server}}(k) / R^{\text{server}}, \quad (4)$$

where R^{server} represents the server computation capability.

The communication time includes both activation upload and gradient download:

$$T_i^{\text{comm}}(k) = S_k / B_i^{\uparrow} + S_k^{(g)} / B_i^{\downarrow}, \quad (5)$$

where $B_i^{\uparrow}$ and $B_i^{\downarrow}$ denote the uplink and downlink bandwidths of client i , respectively.

To avoid selecting excessively shallow or excessively deep cuts, we additionally introduce a balance term that captures the fraction of total computation assigned to the client side:

$$\phi(k) = \text{Comp}^{\text{client}}(k) / (\text{Comp}^{\text{client}}(k) + \text{Comp}^{\text{server}}(k)). \quad (6)$$

Given a target client-side computation ratio $\rho \in [0, 1]$, the balance penalty is defined as

$$P(k) = |\phi(k) - \rho|. \quad (7)$$

Since the computation and communication terms may have different numerical scales, all quantities are normalized across candidate cut layers. The overall cut-layer cost for client i is then defined as

$$\text{CutCost}_i(k) = w_{\text{client}}\hat{T}_i^{\text{client}}(k) + w_{\text{server}}\hat{T}^{\text{server}}(k) + w_{\text{comm}}\hat{T}_i^{\text{comm}}(k) + w_{\text{balance}}\hat{P}(k), \quad (8)$$

where $\hat{T}_i^{\text{client}}(k)$, $\hat{T}^{\text{server}}(k)$, $\hat{T}_i^{\text{comm}}(k)$, and $\hat{P}(k)$ denote the normalized client computation, server computation, communication, and balance penalty terms, respectively. The weights w_{client} , w_{server} , w_{comm} , and w_{balance} control the relative importance of computation, communication, and balance preference.

The personalized optimal cut layer for client i is selected as

$$k_i^* = \arg \min_k \text{CutCost}_i(k). \quad (9)$$

For a system-wide deployment, the global cut layer is determined by averaging the cost across all participating clients:

$$k^* = \arg \min_k \frac{1}{N} \sum_{i=1}^N \text{CutCost}_i(k). \quad (10)$$

This formulation enables the cut-layer decision to adapt to heterogeneous client capabilities while jointly accounting for client computation, server computation, bidirectional communication, and workload balance.

C. Client Clustering

To mitigate the impact of non-IID data while preserving data privacy, clients are grouped based on the similarity of their local data distributions without sharing raw data. Specifically, each client i computes a normalized label distribution vector

$$\mathbf{p}_i = [p_{i1}, p_{i2}, \dots, p_{iC}], \quad (11)$$

where p_{ic} denotes the proportion of samples belonging to class c in the local dataset D_i , and C is the total number of classes.

Each client shares only this statistical summary $\mathbf{p}_i$ with a central coordinator, ensuring that no raw data is exposed. Based on these distributions, the similarity between clients i and j is defined as

$$\text{Sim}(i, j) = \|\mathbf{p}_i - \mathbf{p}_j\|. \quad (12)$$

Using this similarity metric, clients are partitioned into K disjoint clusters $\{\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_K\}$ such that

$$\bigcup_{m=1}^K \mathcal{C}_m = \mathcal{U}, \quad \mathcal{C}_m \cap \mathcal{C}_n = \emptyset, \quad m \neq n. \quad (13)$$

This clustering mechanism groups clients with similar data characteristics, reducing gradient divergence and improving statistical efficiency. Furthermore, clustering provides the foundation for topology-aware intra-cluster communication, enabling structured collaboration among clients through local interactions while maintaining data privacy.

D. Topology-Aware Client Communication

Within each cluster $\mathcal{C}_m$, clients communicate over a pre-defined topology represented as a graph $G_m = (\mathcal{C}_m, E_m)$, where E_m denotes the set of communication links. Typical examples include line, ring, and torus topologies.

Unlike traditional approaches that rely on global aggregation or full client coordination, we introduce topology-aware intra-cluster communication at the client side. After receiving server-side gradients, each client performs local communication with its neighbors to refine its update. Let g_i denote the gradient received by client i from the server with respect to the cut-layer activation. Each client updates its gradient via topology-based mixing:

$$\tilde{g}_i = \sum_{j \in \mathcal{N}(i) \cup \{i\}} w_{ij} g_j, \quad (14)$$

where $\mathcal{N}(i)$ denotes the set of neighbors of client i in G_m , and w_{ij} are mixing weights satisfying $\sum_j w_{ij} = 1$.

The refined gradient $\tilde{g}_i$ is then used for local backpropagation and model updates. This intra-cluster mixing enables distributed gradient refinement through local interactions, reducing gradient variance and improving convergence while avoiding full global synchronization.

The per-round communication cost consists of two components:

$$\text{CommCost}_m = \text{ServerComm}_m + \text{TopologyComm}_m, \quad (15)$$

where ServerComm_m accounts for client-server communication (activations and gradients), and

$$\text{TopologyComm}_m = |E_m| \cdot S_g \quad (16)$$

captures the intra-cluster communication cost, with S_g denoting the gradient size.

E. Total Training Cost

Let $R(G_m, k)$ denote the number of communication rounds required for convergence under topology G_m and cut layer k . For cluster m , let $t_m(k, G_m)$ denote the per-round training cost, which includes client-side computation, server-side computation, client-server communication, and topology-aware intra-cluster communication.

$$\mathcal{T}_{\text{total}} = \sum_{m=1}^K t_m(k, \mathcal{C}_m, G_m) \cdot R(k, \mathcal{C}_m, G_m). \quad (17)$$

This formulation captures the fundamental trade-off in topology-aware split learning. Introducing structured intra-cluster communication may increase the per-round cost $t_m(k, G_m)$ due to additional local interactions among clients. However, these interactions reduce gradient variance and inter-client divergence, which can significantly decrease the number of communication rounds required for convergence.

As a result, the overall training cost may decrease even when the per-round communication overhead increases, provided that the reduction in convergence rounds dominates the additional local communication cost.

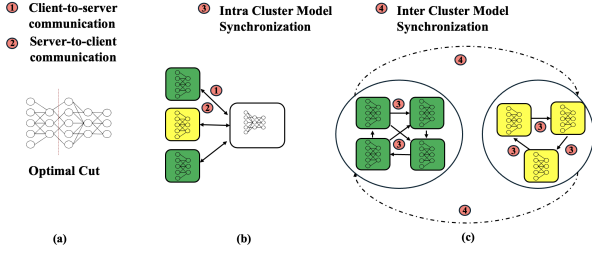


Fig. 2. ToCo-Split: Topology-aware split learning framework, illustrating (a) optimal model cut selection, (b) client-server split training with activation and gradient exchange, and (c) intra- and inter-cluster model synchronization for efficient distributed learning.

F. Problem Formulation

Our objective is to jointly determine the optimal cut layer, client clustering, and communication topology in order to minimize the overall training cost of the split learning system.

These variables are inherently coupled. The cut layer controls the trade-off between computation and communication, clustering influences statistical heterogeneity within groups, and the communication topology affects both local communication overhead and convergence behavior. As a result, improving one component may negatively affect another. For example, deeper cut layers reduce activation transmission but increase client-side computation, while denser topologies improve gradient propagation at the cost of additional local communication. Specifically, the optimization jointly determines: (i) the cut layer k that partitions the model between clients and the server, (ii) the client clusters $\{\mathcal{C}_m\}$ that reduce intra-cluster heterogeneity, and (iii) the communication topologies $\{G_m\}$ that regulate local client interaction. Formally, the optimization problem is defined as

$$\min_{k, \{\mathcal{C}_m\}, \{G_m\}} \mathcal{T}_{\text{total}}(k, \{\mathcal{C}_m\}, \{G_m\}), \quad (18)$$

subject to

$$k \in \{1, 2, \dots, L\}, \quad (19)$$

$$\bigcup_{m=1}^K \mathcal{C}_m = \mathcal{U}, \quad \mathcal{C}_m \cap \mathcal{C}_n = \emptyset, \quad (20)$$

$$G_m = (\mathcal{C}_m, E_m), \quad \forall m = 1, \dots, K. \quad (21)$$

The first constraint ensures a valid cut layer selection, the second enforces a disjoint clustering of all clients, and the third defines a valid communication topology within each cluster. This formulation enables a unified framework that jointly balances computation efficiency, communication overhead, and convergence behavior.

IV. PROPOSED FRAMEWORK

Fig. 2 illustrates the overall pipeline of the proposed topology-aware split learning framework. Rather than treating cut layer selection, clustering, and communication independently, the framework integrates these components into a unified training process.

A. Framework Overview

The training proceeds in four coordinated stages:

- 1) *Cut layer selection*: Prior to training, the optimal cut layer k^* is selected to balance computation and communication cost, determining how the model is partitioned between clients and the server.
- 2) *Client clustering*: Clients are grouped into clusters $\{\mathcal{C}_m\}$ based on the similarity of their data distributions, ensuring that clients within each cluster exhibit similar learning behavior and reduced gradient disagreement.
- 3) *Topology construction*: Within each cluster, a structured communication graph $G_m = (\mathcal{C}_m, E_m)$ is established to regulate local interactions among clients.
- 4) *Iterative training with local interaction*: Training proceeds iteratively by combining split learning with topology-aware client collaboration.

B. Forward and Backward Pipeline

At each communication round, the following steps are executed:

Forward propagation (client $\rightarrow$ server): Each client performs forward propagation up to the cut layer k^* and transmits intermediate activations to the server.

Server-side computation: The server completes the forward pass, computes the loss, and performs backpropagation to obtain gradients with respect to the cut-layer activations.

Gradient distribution: The server sends gradients g_i back to the corresponding clients.

Intra-cluster gradient mixing: Clients refine their gradients through local communication:

$$\tilde{g}_i = \sum_{j \in \mathcal{N}(i) \cup \{i\}} w_{ij} g_j, \quad (22)$$

where $\mathcal{N}(i)$ denotes the neighborhood of client i . This local averaging reduces stochastic fluctuations in gradient estimates.

Inter-cluster alignment (periodic): At predefined intervals, clients incorporate cross-cluster information:

$$\tilde{g}_i^{\text{inter}} = (1 - \alpha) g_i + \alpha g_j, \quad (23)$$

which contracts discrepancies between clusters and prevents divergence.

Local update: Each client performs backpropagation using the refined gradients to update its model parameters.

C. Design Intuition

The proposed pipeline introduces two key improvements over standard split learning:

- *Localized collaboration*: Clients interact within structured neighborhoods instead of relying on global aggregation, improving scalability and communication efficiency.
- *Variance and heterogeneity reduction*: Intra-cluster mixing stabilizes gradient updates, while clustering ensures that interacting clients exhibit similar data characteristics.

Together, these mechanisms improve training stability and accelerate convergence by reducing both gradient variance and inter-client divergence.

D. Convergence Insight and Simplified Bound

We begin from the non-convex convergence results of split federated learning given in [22]. For the full participation setting, two algorithms are analysed: SFL-V1, in which the main server maintains a separate server-side model per client and aggregates them every $\tilde{\tau}$ iterations, and SFL-V2, in which the main server maintains a single shared server-side model updated sequentially by each client.

$$T^{-1} \sum_{t=0}^{T-1} \eta_t \mathbb{E}[\|\nabla f(x^t)\|^2] \leq 4(f(x^0) - f^*) / (T\tau_{\min}) + 8NS(\tau^2 + \tilde{\tau}^2) / (T\tau_{\min}) \sum_{n=1}^N a_n^2(\sigma_n^2 + \epsilon^2) \sum_{t=0}^{T-1} \eta_t^2. \quad (24)$$

where $\tau_{\min} = \min\{\tau, \tilde{\tau}\}$.

$$T^{-1} \sum_{t=0}^{T-1} \eta_t \mathbb{E}[\|\nabla f(x^t)\|^2] \leq 4(f(x^0) - f^*) / (T\tau) + 8NS\tau/T \sum_{n=1}^N (a_n^2 + 1)(\sigma_n^2 + \epsilon^2) \sum_{t=0}^{T-1} \eta_t^2. \quad (25)$$

The structural difference between SFL-V1(24) and SFL-V2(25) reflects the two architectures. In SFL-V1, the noise coefficient depends on a_n^2 and the dual aggregation intervals $(\tau, \tilde{\tau})$, while in SFL-V2 the additional $+1$ term captures the extra noise introduced by the sequential update of a shared server-side model.

a) Simplified convergence rate: To obtain an interpretable form of the convergence behavior, we adopt standard assumptions commonly used in non-convex stochastic optimization. Specifically, we assume that: (i) the step size is constant, i.e., $\eta_t = \eta$, (ii) stochastic gradients are unbiased with bounded variance, and (iii) the deviation between local and global gradients, induced by non-IID data distributions, is bounded. These assumptions ensure that randomness in training and data heterogeneity remain controlled and do not grow unbounded during optimization.

Under these conditions, the detailed bounds in (24) and (25) can be simplified by absorbing system-dependent constants (e.g., number of clients, aggregation intervals, and scaling factors) into a constant term. This yields the dominant convergence behavior:

$$\mathbb{E}[\|\nabla f(x^t)\|^2] = \mathcal{O}((\sigma^2 + \epsilon^2)/T), \quad (26)$$

where σ^2 represents the variability introduced by stochastic gradient estimation (e.g., mini-batch noise), and ϵ^2 quantifies the inconsistency between client updates due to differences in local data distributions.

b) Rounds to convergence: For non-convex objectives, convergence is defined in terms of reaching an approximate stationary point, where the gradient magnitude becomes sufficiently small. Formally, we require:

$$\mathbb{E}[\|\nabla f(x^t)\|^2] \leq \delta^2, \quad (27)$$

where $\delta > 0$ specifies the desired level of optimization accuracy. Substituting (26) into this condition gives:

$$\frac{\sigma^2 + \epsilon^2}{T} \leq \delta^2, \quad (28)$$

which implies that the number of communication rounds required to reach accuracy δ scales as:

$$T = \mathcal{O}((\sigma^2 + \epsilon^2)/\delta^2). \quad (29)$$

This expression shows that convergence is fundamentally governed by two factors: (i) stochastic noise in gradient estimation, and (ii) disagreement between client updates due to heterogeneous data.

c) Implication for our framework: Our framework is designed to explicitly reduce both of these factors. First, intra-cluster gradient mixing performs local averaging of neighboring updates, which smooths stochastic fluctuations and reduces the effective gradient variance. Second, clustering groups clients with similar data distributions, thereby reducing the discrepancy between their gradients.

As a result, the convergence behavior can be expressed as:

$$T_{\text{full}} = \mathcal{O}((\sigma_{\text{eff}}^2 + \epsilon_m^2)/\delta^2), \quad (30)$$

where σ_{eff}^2 denotes the reduced variance after local averaging, and ϵ_m^2 denotes the reduced heterogeneity within clusters. Since $\sigma_{\text{eff}}^2 \leq \sigma^2$ and $\epsilon_m^2 \leq \epsilon^2$, the required number of communication rounds is correspondingly reduced.

This establishes a direct connection between our design choices and improved convergence efficiency.

d) Communication-Computation Trade-off: While our framework introduces additional intra-cluster communication and local processing, resulting in a higher per-round cost compared to the baseline, it significantly reduces the number of communication rounds required for convergence. The total training cost can be expressed as

$$\mathcal{T} = T \cdot t_{\text{round}}, \quad (31)$$

where T denotes the number of rounds and t_{round} represents the per-round cost.

For the baseline method, we have $\mathcal{T}_{\text{base}} = T_{\text{base}} \cdot t_{\text{base}}$, whereas for our framework:

$$\mathcal{T}_{\text{full}} = T_{\text{full}} \cdot (t_{\text{server}} + t_{\text{intra}} + t_{\text{inter}}/K). \quad (32)$$

Although $t_{\text{full}} > t_{\text{base}}$ due to additional intra-cluster communication, our framework reduces the dominant convergence factors σ^2 and ϵ^2 , leading to $T_{\text{full}} \ll T_{\text{base}}$. As a result, the overall training cost is reduced whenever

$$T_{\text{full}} \cdot t_{\text{full}} < T_{\text{base}} \cdot t_{\text{base}}. \quad (33)$$

Importantly, in highly heterogeneous settings, the baseline requires a significantly larger number of rounds due to increased gradient divergence, while our framework stabilizes convergence through clustering and topology-aware mixing. This creates a regime in which the reduction in communication rounds outweighs the additional per-round overhead, leading to improved overall efficiency.

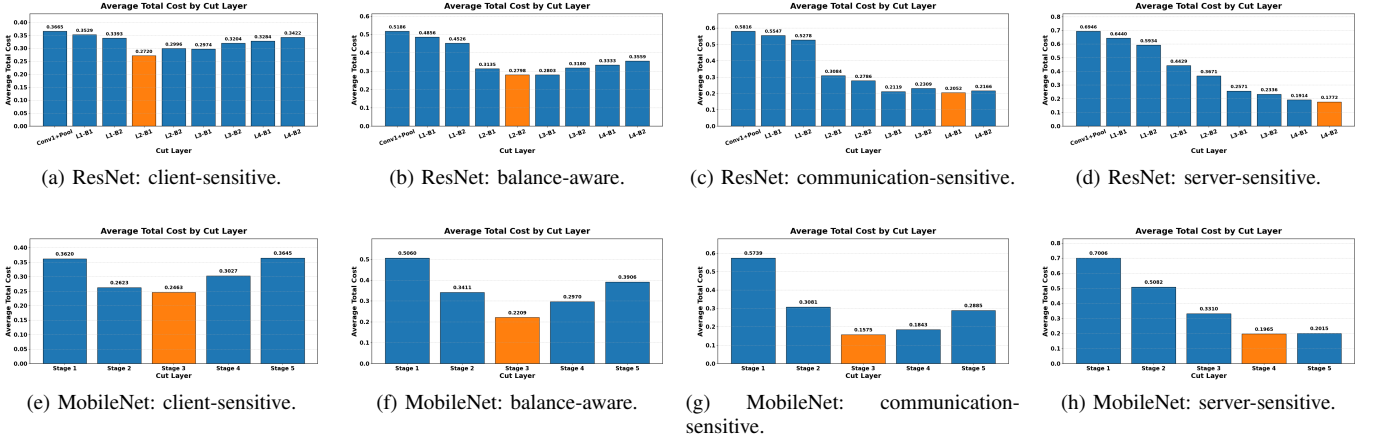


Fig. 3. Sensitivity analysis of the proposed cut-layer selection framework under different weighting configurations for ResNet and MobileNet.

V. EXPERIMENTAL EVALUATION

In this section, we evaluate the proposed topology-aware split learning framework under heterogeneous edge settings. Experiments are conducted on CIFAR-10 and SVHN [24] using ResNet-18 [25], and Mobilenet-V2 [26], with client data partitioned by Dirichlet-based non-IID distributions to model different levels of label skew.

The evaluation focuses on three aspects: (i) cut-layer selection, which analyzes the computation–communication trade-off across candidate split locations; (ii) distribution-aware client grouping, which measures the reduction of intra-cluster heterogeneity; and (iii) topology-aware gradient mixing, which compares line, ring, and torus communication structures against baseline split learning methods. Performance is evaluated using test accuracy, communication overhead, heterogeneity reduction, and overall accuracy–communication trade-off.

A. Cut Layer Sensitivity Analysis

To evaluate the proposed cut-layer optimization framework, we analyze how different weighting configurations influence the selected split location across two representative neural network architectures: ResNet-18 and MobileNet. Specifically, we vary the relative importance of client-side computation, server-side computation, communication overhead, and workload balance within the cut-layer cost formulation.

Fig. 3 illustrates the normalized total cost across different candidate cut layers under four weighting configurations. The top four subfigures report the results for ResNet-18, while the bottom four subfigures report the results for MobileNet. Across both architectures, the proposed formulation adapts the selected cut layer according to the dominant system priority.

For ResNet-18, the client-sensitive configuration selects an intermediate cut around L2-B2, indicating that the framework avoids placing excessive computation on the client while still reducing unnecessary server-side burden. Under the balance-aware setting, the selected cut shifts slightly earlier to L2-B1, reflecting a more even distribution of computation between the

client and server. When communication overhead is emphasized, the optimal cut moves deeper into the network, around L4-B1, because deeper cuts reduce the size of transmitted activations. In the server-sensitive setting, the framework selects the deepest candidate layer, L4-B2, allowing more computation to be completed on the client side and thereby reducing server workload.

For MobileNet, the candidate cut layers are grouped by stage. The results show a similar adaptive behavior, although the optimal choices differ due to the architectural structure of MobileNet. Under client-sensitive and balance-aware configurations, Stage 3 produces the lowest cost, suggesting that an intermediate split provides the best trade-off between client computation and overall system efficiency. In the communication-sensitive setting, Stage 3 is again selected, showing that the activation and computation profile of MobileNet makes the middle stage highly efficient for reducing total cost. Under the server-sensitive configuration, the selected cut shifts deeper to Stage 4, which allows more computation to be handled on the client and reduces the server-side processing burden.

Overall, the results demonstrate that the proposed cut-layer formulation effectively captures the trade-off among communication overhead, client computation, server computation, and workload balance. The framework does not select a fixed split point; instead, it adapts to both the weighting configuration and the architecture-specific computation and activation patterns. This confirms its suitability for heterogeneous edge environments where devices, networks, and server resources may vary significantly.

B. Topology-Aware Client Grouping Analysis

Here, we evaluate topology-aware client grouping before split learning training using a shared ResNet-18 backbone on CIFAR-10 and SVHN. Non-IID clients are generated using Dirichlet partitioning with $\beta \in \{0.05, 0.10, 0.15, 0.20, 0.30, 0.40, 0.50, 0.70, 1.00\}$. A smaller β indicates stronger label skew, while a larger β indicates more balanced client distributions. As expected, global het-

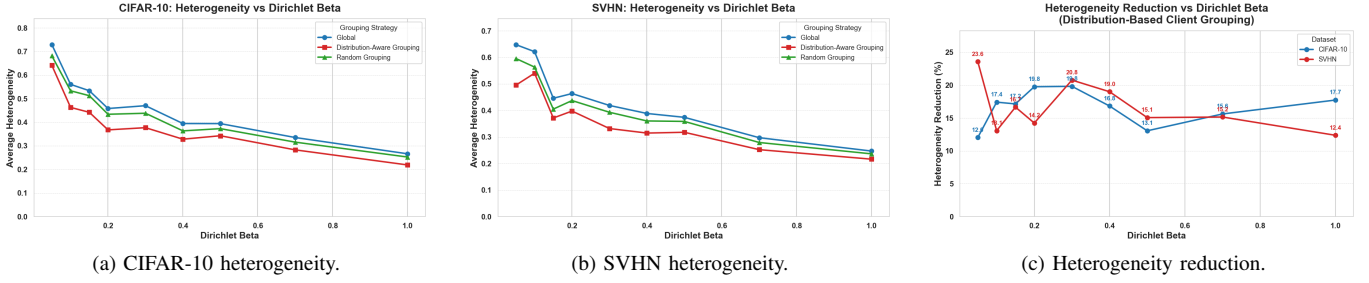


Fig. 4. Effect of distribution-aware client grouping under different Dirichlet non-IID settings on CIFAR-10 and SVHN.

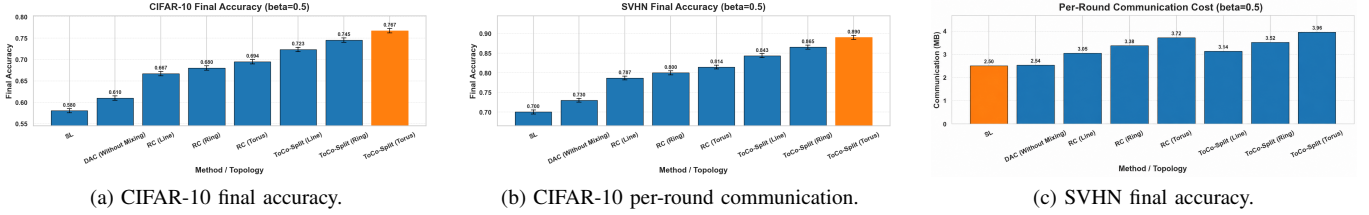


Fig. 5. Full framework comparison for ToCo-Split, Random Clustering (RC), Data Aware Clustering without mixing (DAC), and Split Learning (SL).

TABLE II
ACCURACY AND COMMUNICATION COST ACROSS NON-IID LEVELS

Dataset	β	SL		DAC		RC (Torus)		ToCo-Split (Torus)		ToCo-Split Gain		
		Acc.	Comm. MB	Acc.	Comm. MB	Acc.	Comm. MB	Acc.	Comm. MB	vs. SL	vs. DAC	vs. RC
CIFAR-10	0.1	0.5100	125.0	0.5398	127.0	0.6544	186.0	0.7405	198.0	+23.05 pp	+20.08 pp	+8.61 pp
CIFAR-10	0.5	0.5800	125.0	0.6098	127.0	0.6944	186.0	0.7670	198.0	+18.70 pp	+15.72 pp	+7.26 pp
CIFAR-10	1.0	0.6300	125.0	0.6598	127.0	0.7203	186.0	0.7821	198.0	+15.21 pp	+12.24 pp	+6.18 pp
SVHN	0.1	0.6300	125.0	0.6598	127.0	0.7744	186.0	0.8805	198.0	+25.05 pp	+22.08 pp	+10.61 pp
SVHN	0.5	0.7000	125.0	0.7298	127.0	0.8144	186.0	0.8900	198.0	+19.00 pp	+16.03 pp	+7.56 pp
SVHN	1.0	0.7500	125.0	0.7798	127.0	0.8403	186.0	0.8900	198.0	+14.00 pp	+11.03 pp	+4.97 pp

erogeneity decreases as β increases. For CIFAR-10, ϵ_{global} decreases from 0.7289 at $\beta = 0.05$ to 0.2660 at $\beta = 1.00$. For SVHN, it decreases from 0.6479 to 0.2468.

Fig. 4 shows that distribution-aware grouping consistently reduces heterogeneity compared with the global baseline and random grouping. For CIFAR-10, grouped heterogeneity decreases from 0.7289 to 0.6411 at $\beta = 0.05$ and from 0.4580 to 0.3675 at $\beta = 0.20$, with the best reduction of 19.83% at $\beta = 0.30$. For SVHN, the largest reduction is 23.58% at $\beta = 0.05$, with strong reductions also observed at $\beta = 0.30$ and $\beta = 0.40$, reaching 20.78% and 19.01%, respectively.

Overall, these results show that the proposed distribution-aware grouping reduces statistical heterogeneity before split learning training. This creates more coherent client groups and provides a stronger basis for topology-aware gradient mixing in heterogeneous edge environments.

C. Full Framework Comparison

We next evaluate the full topology-aware split learning framework and compare it with standard split learning (SL), distribution-aware clustering without mixing (DAC), random clustering with topology-based mixing (RC), and the proposed ToCo-Split framework. The experiments are conducted on CIFAR-10 and SVHN using the same ResNet-18 backbone

and the selected cut layer from Subsection V-A. The objective is to evaluate the accuracy–communication trade-off: topology-aware mixing improves learning under non-IID data, but introduces additional communication overhead.

Table II summarizes the performance across different non-IID levels. ToCo-Split consistently achieves the highest accuracy on both datasets for all values of β . The gain is largest under highly skewed data distributions. At $\beta = 0.1$, ToCo-Split improves accuracy over SL by 23.05 percentage points on CIFAR-10 and 25.05 percentage points on SVHN. Compared with DAC, ToCo-Split still improves accuracy by 20.08 and 22.08 percentage points, respectively.

As the data distribution becomes more balanced, the relative gain decreases. At $\beta = 1.0$, ToCo-Split improves over SL by 15.21 percentage points on CIFAR-10 and 14.00 percentage points on SVHN. This trend shows that topology-aware collaboration is most useful when client updates are highly divergent. In lower- β settings, clients observe more heterogeneous local distributions, so torus-based mixing improves update consistency and provides stronger accuracy gains.

The communication results show the cost of this improvement. SL requires 125.0 MB total communication, while DAC requires 127.0 MB. RC with torus requires 186.0 MB, and ToCo-Split requires 198.0 MB due to inter-client topology

communication. Although ToCo-Split has the highest communication cost, it also provides the largest accuracy improvement. Its gain over RC remains 8.61 percentage points on CIFAR-10 and 10.61 percentage points on SVHN at $\beta = 0.1$, showing that data-aware clustering and topology mixing are complementary.

Overall, ToCo-Split offers the strongest accuracy–communication trade-off when data heterogeneity is high. While its communication cost is higher than SL, DAC, and RC, the added topology-aware communication produces substantial accuracy gains under low- β distributions. As the data approaches IID, the marginal benefit decreases, suggesting that topology-aware mixing is most valuable for statistically heterogeneous split learning environments.

VI. CONCLUSION AND FUTURE WORK

In this paper, we presented ToCo-Split, a topology-aware split learning framework for efficient edge intelligence. The framework jointly considers cut-layer selection, distribution-aware client grouping, and structured client-side communication to improve learning under heterogeneous and non-IID settings. Our analysis shows that client clustering reduces statistical heterogeneity, while topology-aware gradient mixing improves update consistency among neighboring clients. Experimental results on CIFAR-10 and SVHN demonstrate that ToCo-Split achieves higher accuracy than standard split learning, distribution-aware clustering alone, and random topology-based mixing. The results also reveal an important trade-off: denser topologies improve accuracy but introduce additional per-round communication overhead, making it most beneficial in highly non-IID settings, where the gain from reducing gradient disagreement outweighs the extra communication cost. As future work, we plan to investigate hypercube-based topologies for faster information propagation [27] in larger client groups and study pairwise gradient mixing strategies that reduce communication. Furthermore, the impact of inter and intra-cluster communication frequency, early stopping to preserve bandwidth, and the convergence is to be analyzed under larger models and a more realistic edge environment.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agueria y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [2] M. Ye, X. Fang, B. Du, P. C. Yuen, and D. Tao, “Heterogeneous federated learning: State-of-the-art and research challenges,” *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–44, 2023.
- [3] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, “Split learning for health: Distributed deep learning without sharing raw patient data,” *arXiv preprint arXiv:1812.00564*, 2018.
- [4] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, “Splitfed: When federated learning meets split learning,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 36, no. 8, 2022, pp. 8485–8493.
- [5] X. Cao, T. Başar, S. Diggavi, Y. C. Eldar, K. B. Letaief, H. V. Poor, and J. Zhang, “Communication-efficient distributed learning: An overview,” *IEEE journal on selected areas in communications*, vol. 41, no. 4, pp. 851–873, 2023.
- [6] A. Nedic and A. Ozdaglar, “Distributed subgradient methods for multi-agent optimization,” *IEEE Transactions on automatic control*, vol. 54, no. 1, pp. 48–61, 2009.

- [7] P. Neal, C. Eric, P. Borja, and E. Jonathan, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends® in Machine learning*, vol. 3, no. 1, pp. 1–122, 2011.
- [8] J. Konečný, H. B. McMahan, D. Ramage, and P. Richtárik, “Federated optimization: Distributed machine learning for on-device intelligence,” *arXiv preprint arXiv:1610.02527*, 2016.
- [9] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, “Qsgd: Communication-efficient sgd via gradient quantization and encoding,” *Advances in neural information processing systems*, vol. 30, 2017.
- [10] S. U. Stich, J.-B. Cordonnier, and M. Jaggi, “Sparsified sgd with memory,” *Advances in neural information processing systems*, vol. 31, 2018.
- [11] N. H. Tran, W. Bao, A. Zomaya, M. N. Nguyen, and C. S. Hong, “Federated learning over wireless networks: Optimization model design and analysis,” in *IEEE INFOCOM 2019-IEEE conference on computer communications*. IEEE, 2019, pp. 1387–1395.
- [12] N. Jia, Z. Qu, B. Ye, Y. Wang, S. Hu, and S. Guo, “A comprehensive survey on communication-efficient federated learning in mobile edge environments,” *IEEE Communications Surveys & Tutorials*, vol. 27, no. 6, pp. 3710–3741, 2025.
- [13] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, “Scaffold: Stochastic controlled averaging for federated learning,” in *International conference on machine learning*. PMLR, 2020, pp. 5132–5143.
- [14] Z. Lin, G. Qu, X. Chen, and K. Huang, “Split learning in 6g edge networks,” *IEEE Wireless Communications*, vol. 31, no. 4, pp. 170–176, 2024.
- [15] O. Gupta and R. Raskar, “Distributed learning of deep neural network over multiple agents,” *Journal of Network and Computer Applications*, vol. 116, pp. 1–8, 2018.
- [16] Z. Lin, G. Zhu, Y. Deng, X. Chen, Y. Gao, K. Huang, and Y. Fang, “Efficient parallel split learning over resource-constrained wireless edge networks,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 10, pp. 9224–9239, 2024.
- [17] S. Lee, T. Zhang, and A. S. Avestimehr, “Layer-wise adaptive model aggregation for scalable federated learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 7, 2023, pp. 8491–8499.
- [18] J. Kim, S. Kang, and S. Lee, “Layer-wise update aggregation with recycling for communication-efficient federated learning,” *arXiv preprint arXiv:2503.11146*, 2025.
- [19] M. Marinova, M. Poposka, Z. Hadzi-Velkov, and V. Rakovic, “Optimal cut layer bounds for split learning,” *IEEE Communications Letters*, 2025.
- [20] J. Dachille, C. Huang, and X. Liu, “The impact of cut layer selection in split federated learning,” *arXiv preprint arXiv:2412.15536*, 2024.
- [21] Y. Duan, X. Li, and J. Wu, “Topology design and graph embedding for decentralized federated learning,” *Intelligent and Converged Networks*, vol. 5, no. 2, pp. 100–115, 2024.
- [22] P. Han, C. Huang, G. Tian, M. Tang, and X. Liu, “Convergence analysis of split federated learning on heterogeneous data,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 103 476–103 544, 2024.
- [23] J. Shen, N. Cheng, X. Wang, F. Lyu, W. Xu, Z. Liu, K. Aldubaikhy, and X. Shen, “Ringsfl: An adaptive split federated learning towards taming client heterogeneity,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5462–5478, 2023.
- [24] A. Krawczyk and A. Gepperth, “An analysis of best-practice strategies for replay and rehearsal in continual learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 4196–4204.
- [25] R. K. Bhogal and A. Singh, “A comprehensive review of resnet-18: Architecture and applications,” in *International Conference on Advances in Data-driven Computing and Intelligent Systems*. Springer, 2024, pp. 595–607.
- [26] A. Michele, V. Colin, and D. D. Santika, “Mobilenet convolutional neural networks and support vector machines for palmprint recognition,” *Procedia Computer Science*, vol. 157, pp. 110–117, 2019.
- [27] N. F. Prangon and J. Wu, “Beyond the teacher-student paradigm: The role of network structure in knowledge distillation,” in *Proc. of the International Conference on Computing, Networking and Communications (ICNC)*, 2026.